

All You Need Are Programmable Docs

The Vision (idée fixe)

Most behaviours, apps, tools – so I think – would be easier to develop, adapt, and integrate if they were expressed as cells and formulas in a programmable doc. The potential of spreadsheet-like docs is far from exhausted.

I envision one huge, versioned, collaborative doc that becomes the de facto operating system of my computer. The doc forms a tree (not a grid) and becomes the new location for most data -- from the system internals, to the user's personal files, clipboard contents, selections, and more. It is cells and formulas all the way down and the SPU – spreadsheet processing unit – hardware makes it fast.

Designing an “SPU” and an OS is a bit too much, so I am now designing something smaller with a more specific use-case in mind. Still, the vision helps guide the design, for it distills various ideas from my earlier work that have worked quite well.

The Goal

The specific thing I am designing is a programmable doc that can serve as a malleable, shared “control room” for any project – a tool I personally miss. As a team member at various projects, I have repeatedly experienced frustration with scattered and poorly linked information, broken automations etc. This work is an attempt to fix this.

Takeaways from an earlier prototype

This work is a continuation of my earlier work [A Non-Tabular Spreadsheet With Broad Applicability](#) . Also, here is a [demo video](#) that illustrates some aspects of the prototype.

- 1) Reifying as much state as possible into the document (the view state of all editor instances, cursor position, selection, clipboard) empowers the user. For example, it enables her to
 - a) build tools via formulas without leaving the system,
 - b) save and re-apply the UI state or its parts by directly manipulating cells that reify the UI state.
- 2) The concept of self-healing works. Less invariants means less “why can’t I do this?” moments. It also seems to overall simplify the substrate implementation. The view/projection gets more complicated, but in a way that is relatively easy to design and reason about. (This is important in scenarios where other processes can modify the model in any way at any time. This may sound “naive”, but this integration model – a directory the user can manipulate through several tools concurrently – is very easy to grasp and doesn’t get in the user’s way.)

Desirable properties

Here are some properties I consider desirable:

- A document can become an app. An empty document can be continuously evolved into a full-featured application – without resorting to any sort of extensions in a “regular” programming language. As explored in the previous prototype, the tools to *develop* the doc can also be part of the doc!
- The doc format should be able to express hierarchies, should allow the user to switch between different projections of the same data, and crucially, should have a sufficiently powerful formula language.
- It’s a substrate in the following sense: Its mechanisms are not too domain-specific and can be combined freely, allowing emergent behaviour, and thus a broad range of unexpected use-cases.
- Double down on the pervasive reification and self-healing concepts that have shown great promise in the previous prototype.
- It offers tight integration with the desktop environment. The desktop is still indispensable in many workflows. In particular, it must be possible to embed or link entire files and directories into any cell. Also, embedded files must remain accessible via the filesystem, allowing the user to keep using existing file-based desktop software.
- Collaboration, version control, synchronisation etc. are part of the design, not an afterthought.
 - Allow the user to commit and merge deliberately and manually. The first version will be restricted to this manual version management. Later, “agents” can be added that perform these actions on behalf of the user.
 - Avoid problematic parts of Git’s semantics. Solutions exist. For example, Pijul makes rebase and merge the same thing by using change-based semantics as opposed to snapshot-based semantics.
 - Support links that point to a particular commit in an external Git repository.
- It is closer to Excel than general-purpose programming languages in terms of usability by non-programmers.
- Near-zero boilerplate onboarding: create a doc, share it, start adding data and formulas—no setup needed.
- It must be sufficiently flexible that moving all the project’s data, resources and automation into the document becomes the preferred way.
- The system is local-first – it must remain usable just fine when offline.

The Design

The current design is still evolving and has not yet been implemented, but my previous system “Nezt” does have a prototype implementation.

The document is formed by a hierarchy of cells and is represented as a specifically structured directory on the local filesystem. It will be edited mainly through a dedicated desktop application.

A cell has some metadata, and a content. The metadata specifies the placement of the cell in the hierarchy and other things. The content is any set of files and directories. This ability to embed arbitrary files and directories into the document is intended to make tight integration with the desktop environment easier. Specifically, the user should be able to directly edit those embedded files and directories using existing authoring applications be it VS Code, Adobe Illustrator, or other professional software.

In the ideal case, the team embeds all their digital assets into the document. In some cases, embedding won't be desirable, so there will be various ways to link to external content. For example, it will be possible to refer to a particular commit in a git repository, or to refer to an online resource via an URL. If desired, references may be configured to assert a particular cryptographic hash of the target data. In this case, if the hashes don't match, the linked content is considered invalid until the user confirms that the change is OK.

All kinds of content, embedded or linked, will also be allowed as inputs in formulas. For now, formulas are represented as syntax trees using the hierarchy itself. It is easier to provide a specialised projection than to have to deal with textual notations, grammars and parsing. Working with trees directly does have downsides, but it also makes a lot of problems just disappear.

As in my earlier prototype “Nezt”, a formula returns a dynamically constructed document fragment. This can be a single value or, say, an entire hierarchically structured report. It is desirable to allow references to target cells in such generated hierarchies, but it is not clear how to keep the target cell identities stable. This is one of the research problems that will need to be solved.

UI state like cursor position, current selection, clipboard content, and hierarchy folding are all reified as cell content in a special part of the document. As I have demonstrated in “Nezt”, this reification allows the user to easily build various tools without having to leave the environment. Honestly, the reification feels magical. For example, you can save and restore the UI state or its parts by just copy-pasting material between the special and some other part of the doc.

Unlike the old prototype, the new one will be designed to allow collaboration and versioning based on CRDTs. A lot of design and research work is still needed here.

Initially, users will explicitly invoke versioning actions like commit, merge, etc. – there will be no real-time collaboration.

Given the nature of the project, I believe extensive dogfooding as soon as possible is a must.

The Problems

What is a suitable way to merge reorderings in lists? Many list CRDTs only support add and delete operations. This is not sufficient, because often parts of the document will form a “story” to read or a layout to look at, and authors need to reshuffle elements while crafting those parts.

How to treat undo in the presence of versioning? I probably don’t want each keystroke to become a commit! What if I undo past a commit? Does it change the version I am at? What if I undo past a merge action?

What if I change an embedded file in an external tool and then do an undo in the main document editor? Should that change be subject to undo?

How to usefully derive the identity of a cell in a derived tree? If I create a reference to a cell in a derived tree, and a change in formula or its inputs changes the derived tree structure, I want the reference to still point to the cell that the user intuitively considers “the same cell”. Plain paths don’t have this property.

How to make the doc act as a web API?

Resources (to be expanded and improved)

This work is a continuation of my earlier work:

[A Non-Tabular Spreadsheet With Broad Applicability](https://www.researchgate.net/publication/326141821_A_non-tabular_spreadsheet_with_broad_applicability):

https://www.researchgate.net/publication/326141821_A_non-tabular_spreadsheet_with_broad_applicability .

A [demo video](https://youtu.be/pphY94boHq4?si=Sld9SpfUvqgkShwx):

<https://youtu.be/pphY94boHq4?si=Sld9SpfUvqgkShwx>

I think a lot can be achieved by just carefully integrating existing ideas. Here is an incomplete list of systems that have especially shaped my intuitions:

- JetBrains MPS
- Andrea diDessa’s Boxer
- Spreadsheets
- Symbolics Genera
- Oberon
- Smalltalk
- Coda (<https://coda.io>)

Here is an online version of this doc:

<https://docs.google.com/document/d/14ENBXmZe6LT7DDNppiTo4JwUjWEUA0fooi7HV5zqvJk/edit?usp=sharing>

AUTHORS: Pavel Bažant

TITLE: All You Need Are Programmable Docs

+++++++ REVIEW 1 (Gilad Bracha) +++++++

I tend to agree with the title (and yes, it does have the flavor of an idee fixe - that's fine as long as we have some self-awareness and recognize that).

I am very keen on documents that become apps. Where I become skeptical is this idea that conventional programming language is not needed. Even in the age of AI, a real PL underneath is very important. It helps the AI as well as humans, it is essential for precision, repeatability and efficiency. I've also found that it is often useful to build docu-apps on the basis of components built by more conventional programming.

All in favor of local-first, and of extensive dog fooding.

I think having each keystroke be a commit is fine in some circumstances. We experimented with that in an earlier version of Newspeak - a separate repo maintained character-by-character edits between "accepts" of entire methods. So it was just a short term thing, between explicit user "commits" of entire methods. The entire undo system was separate from the "real" repo where more conventional commits were made.

+++++++ REVIEW 3 (Tom Larkworthy) +++++++

The title resonated with me as I, an experienced programmer, have also yearned for programmable docs, and personally found a close match in Observable Notebooks.

"You" in this context for me, is "me", but how generally applicable is this? The author lists some desirable properties of a rich programmable document system. The list is reasonable but I am left wondering what kinds of tasks this system would excel at. Clear examples would be helpful to evaluate whether the design choices support it. In several cases I am left wondering on the motivation for choices.

For example, spreading files on a directory is great for cross tool malleability, yet the author hints at aiming to do everything in the one tool. This tool must clearly be specialized, but this design choice now binds the serialized format to the viewer tool. The pile of files now inhibits sharing, because 1. multiple files are unwieldy to transfer, and 2. also because the receiver needs the specialized software to read it. So now what was the point of scattering the files over a directory format? It has hurt dissemination perhaps. But perhaps I don't understand what the format will excel at.

The reification and CRDTs sounds intellectually interesting but there is not enough explanation to understand. I suspect CRDTs and SOTA collaboration experience in particular are risky scope expansions, maybe they need to be motivated better to explain why they are so fundamentally important.

I am left intrigued by the work, the title is very compelling, I am greatly looking forward to seeing the progress.

+++++++ REVIEW 4 (Steven Githens) +++++++

I really like the vision and goal here and several of the desirable properties, most especially the "document can become an app". As my current work is on the modernization of Andy diSessa's Boxer, I'd like to think that your document is just always the app. Additionally I would hope that the 2nd and 3rd Desirable properties of bullets get a lot of attention for any system we were to call a Substrate.

For my category of users, I'm less concerned about version control and these types of things, I hope to keep these things very simple and document oriented, but I'm open to them being possible if we could someone find ways of manipulating versioning and the UI for it that made some sort of sense to less technical users. Even the

somewhat userfriendly interfaces of the macOS Timemachine and time sliders on Google Docs and other editors tend to baffle a lot of the population.